

# MiniPaint

## Documentação Completa do Projeto

### Computação Gráfica — Curvas Paramétricas

Linguagem: Java (Swing) • JDK 8+ • Junho 2025

#### Sumário

1. Visão Geral do Projeto
2. Organização do Código
  1. Estrutura de Arquivos
  2. Hierarquia de Classes
  3. Enum ToolMode
  4. Fluxo de Interação (CanvasPanel)
3. Modelos Matemáticos
  1. Rasterização de Retas (DDA e Bresenham)
  2. Rasterização de Círculo (Bresenham)
  3. Clipping de Linhas
  4. Curva de Bézier Cúbica
  5. Curva de Hermite Cúbica
  6. Relação Bézier ↔ Hermite
  7. Transformações Geométricas
4. Manual de Uso
  1. Compilar e Executar
  2. Barra de Ferramentas

3. Desenhar Ponto
4. Desenhar Reta
5. Desenhar Círculo
6. Desenhar Polígono
7. Desenhar Curva de Bézier
8. Desenhar Curva de Hermite
9. Seleção e Edição
10. Transformações
11. Atalhos de Teclado

# 1. Visão Geral do Projeto

---

O **MiniPaint** é uma aplicação desktop escrita em **Java Swing** para criação e manipulação de formas geométricas 2D. Foi desenvolvido como projeto prático de **Computação Gráfica**, implementando os principais algoritmos da disciplina:

- **Rasterização** — DDA e Bresenham (retas), Bresenham (círculos)
- **Clipping** — Cohen-Sutherland e Liang-Barsky
- **Transformações 2D** — Translação, Rotação, Escala e Reflexão
- **Curvas Paramétricas** — Bézier Cúbica e Hermite Cúbica

## Requisitos

Java JDK 8 ou superior. Nenhuma dependência externa é necessária.

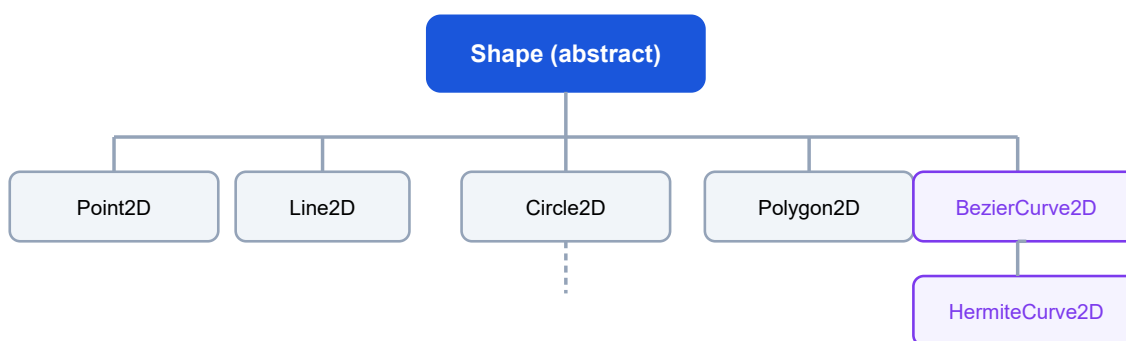
# 2. Organização do Código

---

## 2.1 Estrutura de Arquivos

| Arquivo                       | Responsabilidade                                                                                                                                                                                                                                                                                                                                                                                                           | Linhas |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <code>PaintApp.java</code>    | Ponto de entrada ( <code>main</code> ), cria o <code>JFrame</code> , monta a toolbar com os botões de ferramenta e os toggles de algoritmo (DDA/Bresenham, Cohen/Liang).                                                                                                                                                                                                                                                   | ~150   |
| <code>CanvasPanel.java</code> | Painel de desenho ( <code>JPanel</code> ). Gerencia toda a interação do mouse (desenho, seleção, drag, rotação, escala), mantém a lista de <code>Shape</code> s e realiza a pintura via <code>paintComponent</code> .                                                                                                                                                                                                      | ~760   |
| <code>Definitions.java</code> | Define o enum <code>ToolMode</code> e toda a hierarquia de shapes: <code>Shape</code> (abstrata), <code>Point2D</code> , <code>Line2D</code> , <code>Circle2D</code> , <code>Polygon2D</code> , <code>BezierCurve2D</code> , <code>HermiteCurve2D</code> . Cada shape implementa <code>draw</code> , <code>translate</code> , <code>rotate</code> , <code>scale</code> , <code>clone</code> e <code>isFullyInside</code> . | ~870   |
| <code>Utils.java</code>       | Utilitários de renderização: constantes de cor/stroke e o método <code>prepareShapeGraphics</code> que configura o <code>Graphics2D</code> conforme o estado (normal, selecionado, preview).                                                                                                                                                                                                                               | ~24    |
| <code>fonts/</code>           | Diretório contendo a fonte <code>FontAwesome</code> ( <code>fa-solid-900.otf</code> ) usada nos ícones da toolbar.                                                                                                                                                                                                                                                                                                         | —      |

## 2.2 Hierarquia de Classes



Métodos abstratos de Shape:

```

draw(Graphics, offsetX, offsetY, highlight, app, vLeft, vTop, vRight, vBottom)
translate(dx, dy) | rotate(center, angle) | scale(center, scaleX, scaleY)
isFullyInside(rect, offsetX, offsetY) | clone()
+ Auxiliares: isPolygon(), isPoint(), rotatePoint(px, py, center, angle)

```

## 2.3 Enum ToolMode

| Modo         | Descrição                                                          | Botão |
|--------------|--------------------------------------------------------------------|-------|
| DRAW_POINT   | Desenha pontos individuais                                         | ⊙     |
| DRAW_LINE    | Desenha reta por arraste (press → drag → release)                  | —     |
| DRAW_CIRCLE  | Desenha círculo por arraste (centro → raio)                        | ●     |
| DRAW_POLYGON | Desenha polígono por cliques (fecha ao clicar perto do 1º vértice) | □     |
| DRAW_BEZIER  | Desenha curva de Bézier cúbica (4 cliques)                         | Béz   |
| DRAW_HERMITE | Desenha curva de Hermite cúbica (click-drag-click-drag)            | Herm  |
| SELECT       | Seleção retangular de shapes                                       | ↻     |
| EDIT         | Edição (arraste) de shapes seleccionadas                           | —     |
| PAN          | Navegação (arrastar o canvas)                                      | 👉     |
| ROTATE       | Rotação de shapes seleccionadas por arraste                        | ↻     |
| SCALE        | Escala de shapes seleccionadas por arraste                         | ↗     |

## 2.4 Fluxo de Interação (CanvasPanel)

O `CanvasPanel` mantém um `MouseAdapter` com os handlers:

| Evento                     | Ação                                                                                                 |
|----------------------------|------------------------------------------------------------------------------------------------------|
| <code>mousePressed</code>  | Inicia desenho/seleção/transformação conforme modo atual                                             |
| <code>mouseDragged</code>  | Atualiza preview (retas, círculos, tangentes Hermite) ou aplica transformação (pan, rotação, escala) |
| <code>mouseMoved</code>    | Atualiza preview de polígono, Bézier (pontos parciais) e hint de fechamento                          |
| <code>mouseReleased</code> | Finaliza forma, confirma seleção, ou aplica transformação permanente                                 |
| <code>keyPressed</code>    | V valida polígono aberto; X cancela/exclui                                                           |

O método `paintComponent` desenha, nesta ordem:

1. Todas as shapes finalizadas ( `shapes` )
2. Shapes temporárias de transformação ( `tempShapes` )

3. Preview do shape sendo construído ( `previewShape` )
4. Preview de polígono ( `polygonPreviewLine` )
5. Preview de Hermite ( `hermitePreview` )
6. Pontos de controle em construção (Bézier/Hermite)
7. Hint de fechamento de polígono
8. Retângulo de seleção

## 3. Modelos Matemáticos

### 3.1 Rasterização de Retas

#### 3.1.1 Algoritmo DDA (Digital Differential Analyzer)

O DDA calcula os pixels de uma reta de  $(x_1, y_1)$  a  $(x_2, y_2)$  usando aritmética de ponto flutuante:

```
steps = max(|dx|, |dy|)
x_incr = dx / steps
y_incr = dy / steps

Para i = 0 até steps:
    plot(round(x), round(y))
    x += x_incr
    y += y_incr
```

##### Complexidade

$O(\max(|dx|, |dy|))$  — um pixel por iteração. Usa divisão de ponto flutuante apenas uma vez (no início).

#### 3.1.2 Algoritmo de Bresenham

O algoritmo de Bresenham evita aritmética de ponto flutuante completamente, usando apenas operações inteiras. Utiliza uma **variável de decisão**  $p$  para decidir se o próximo pixel sobe ou não:

```
// Caso |slope| ≤ 1 (shallow):
p = 2·dy - dx
Para i = 0 até dx:
    x += x_incr
    Se p < 0:
        p += 2·dy
    Senão:
        y += y_incr
        p += 2·(dy - dx)
    plot(x, y)
```

##### Derivação da variável de decisão

A decisão baseia-se em qual pixel (E ou NE) está mais próximo da reta real. Definindo  $d = F(x+1, y+\frac{1}{2})$  onde  $F$  é a equação implícita da reta, o sinal de  $d$  determina a escolha. A atualização incremental evita recalcular  $F$  a cada passo.

## 3.2 Rasterização de Círculo (Bresenham)

Utiliza a **simetria de 8 octantes**: calcula apenas o arco de  $0^\circ$  a  $45^\circ$  e replica por espelhamento. A variável de decisão:

```
x = 0, y = r
p = 3 - 2·r

Enquanto x < y:
    x++
    Se p < 0:
        p += 4·x + 6
    Senão:
        p += 4·(x - y) + 10
    y--
    drawSymmetric(cx, cy, x, y) // desenha 8 pontos
```

A função `symmetricalDraw` plota os 8 pontos simétricos:  $(cx \pm x, cy \pm y)$  e  $(cx \pm y, cy \pm x)$ .

## 3.3 Clipping de Linhas

### 3.3.1 Cohen-Sutherland

Cada extremidade da reta recebe um **outcode** de 4 bits (LEFT, RIGHT, TOP, BOTTOM):

| Bit   | Condição      | Região   |
|-------|---------------|----------|
| 0 (1) | $x < vLeft$   | Esquerda |
| 1 (2) | $x > vRight$  | Direita  |
| 2 (4) | $y < vTop$    | Acima    |
| 3 (8) | $y > vBottom$ | Abaixo   |

- **OR = 0**: ambos dentro → aceita trivialmente
- **AND ≠ 0**: ambos no mesmo lado externo → rejeita trivialmente
- Caso contrário: recorta na borda correspondente e recalcula

### 3.3.2 Liang-Barsky

Parametriza a reta como  $P(t) = P_1 + t \cdot (P_2 - P_1)$  com  $t \in [0, 1]$  e calcula os parâmetros  $t$  de entrada/saída para cada borda da janela:

$$p \cdot t \leq q, \text{ onde } p = -dx, q = x_1 - x_{\min} \text{ (borda esquerda)}$$

O algoritmo mantém  $t_i$  (máximo dos de entrada) e  $t_2$  (mínimo dos de saída). Se  $t_i > t_2$ , a reta está fora.

## 3.4 Curva de Bézier Cúbica

A curva de Bézier de grau 3 é definida por **4 pontos de controle**  $P_0, P_1, P_2, P_3$ :

$$P(t) = (1-t)^3 \cdot P_0 + 3 \cdot t \cdot (1-t)^2 \cdot P_1 + 3 \cdot t^2 \cdot (1-t) \cdot P_2 + t^3 \cdot P_3, \quad t \in [0, 1]$$

Os coeficientes são os **Polinômios de Bernstein** de grau 3:

| Polinômio | Fórmula                   | B(0) | B(1) |
|-----------|---------------------------|------|------|
| $B_0(t)$  | $(1-t)^3$                 | 1    | 0    |
| $B_1(t)$  | $3 \cdot t \cdot (1-t)^2$ | 0    | 0    |
| $B_2(t)$  | $3 \cdot t^2 \cdot (1-t)$ | 0    | 0    |
| $B_3(t)$  | $t^3$                     | 0    | 1    |

### Forma Matricial

$$P(t) = [t^3 \quad t^2 \quad t \quad 1] \cdot M_{\text{bézier}} \cdot [P_0 \quad P_1 \quad P_2 \quad P_3]^T$$

$$M_{\text{bézier}} = \begin{vmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{vmatrix}$$

### Propriedades

- **Interpolação:**  $P(0) = P_0, P(1) = P_3$  — a curva passa pelos extremos.
- **Tangentes:**  $P'(0) = 3 \cdot (P_1 - P_0), P'(1) = 3 \cdot (P_3 - P_2)$ .
- **Envoltória convexa:** a curva está sempre dentro do polígono convexo formado pelos 4 pontos.
- **Invariância afim:** transformar os pontos de controle é equivalente a transformar a curva.
- **Diminuição de variação:** a curva não oscila mais que o polígono de controle.

### Implementação no MiniPaint

```
// BezierCurve2D.evalX(t):
double u = 1 - t;
return u*u*u*px[0] + 3*t*u*u*px[1] + 3*t*t*u*px[2] + t*t*t*px[3];
```

A curva é renderizada avaliando  $P(t)$  para  $t = 0, 0.01, 0.02, \dots, 1.0$  (100 passos) e conectando pontos consecutivos com segmentos `Line2D`. Isso garante que o **clipping** e a **rasterização** existentes (DDA/Bresenham, Cohen-Sutherland/Liang-Barsky) se apliquem automaticamente a cada segmento.

### Visualização

Além da curva, o MiniPaint exibe:

- O **polígono de controle**  
(linhas tracejadas cinza entre  $P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow P_3$ )
- **Pontos de controle**  
: verdes ( $P_0, P_3$  — extremos) e laranjas ( $P_1, P_2$  — intermediários)

## 3.5 Curva de Hermite Cúbica

A curva de Hermite é definida por **2 pontos extremos** ( $P_1, P_4$ ) e **2 vetores tangente** ( $R_1, R_4$ ):

$$P(t) = h_1(t) \cdot P_1 + h_2(t) \cdot P_4 + h_3(t) \cdot R_1 + h_4(t) \cdot R_4, \quad t \in [0, 1]$$

Os 4 polinômios base de Hermite são:

| Polinômio | Fórmula           | Significado                                      |
|-----------|-------------------|--------------------------------------------------|
| $h_1(t)$  | $2t^3 - 3t^2 + 1$ | $h_1(0)=1, h_1(1)=0 \rightarrow$ interpola $P_1$ |
| $h_2(t)$  | $-2t^3 + 3t^2$    | $h_2(0)=0, h_2(1)=1 \rightarrow$ interpola $P_4$ |
| $h_3(t)$  | $t^3 - 2t^2 + t$  | $h_3'(0)=1 \rightarrow$ tangente em $P_1 = R_1$  |
| $h_4(t)$  | $t^3 - t^2$       | $h_4'(1)=1 \rightarrow$ tangente em $P_4 = R_4$  |

### Forma Matricial

$$P(t) = [t^3 \quad t^2 \quad t \quad 1] \cdot M_{\text{hermite}} \cdot [P_1 \quad P_4 \quad R_1 \quad R_4]^T$$

$$M_{\text{hermite}} = \begin{vmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{vmatrix}$$

## Propriedades

- **Interpolação completa:**  $P(0) = P_1$ ,  $P(1) = P_4$  — a curva passa por ambos.
- **Tangentes prescritas:**  $P'(0) = R_1$ ,  $P'(1) = R_4$  — controle direto da direção em cada extremo.
- **Continuidade  $C^1$ :** ao encadear curvas com  $R_{4i} = R_{i+1}$ , obtém-se continuidade de tangente.

## Implementação no MiniPaint

```
// HermiteCurve2D.evalX(t):
double t2 = t * t, t3 = t2 * t;
double h1 = 2*t3 - 3*t2 + 1; // interpolação P1
double h2 = -2*t3 + 3*t2;    // interpolação P4
double h3 = t3 - 2*t2 + t;    // tangente R1
double h4 = t3 - t2;          // tangente R4
return h1*p1x + h2*p4x + h3*r1x + h4*r4x;
```

A renderização segue o mesmo padrão da Bézier: 100 segmentos `Line2D`.

### Visualização

O MiniPaint exibe:

- **Tangente  $R_1$**   
: seta tracejada  
**azul**  
partindo de  $P_1$
- **Tangente  $R_4$**   
: seta tracejada  
**laranja**  
partindo de  $P_4$
- **Pontos extremos**  
: discos  
**verdes**

## 3.6 Relação Bézier ↔ Hermite

Toda curva de Hermite pode ser convertida em uma curva de Bézier equivalente:

| Ponto Bézier      | Fórmula         |
|-------------------|-----------------|
| $P_{0\text{béz}}$ | $P_1$           |
| $P_{1\text{béz}}$ | $P_1 + R_1 / 3$ |
| $P_{2\text{béz}}$ | $P_4 - R_4 / 3$ |
| $P_{3\text{béz}}$ | $P_4$           |

#### Demonstração

Da tangente da Bézier:  $P'(0) = 3 \cdot (P_{1\text{béz}} - P_{0\text{béz}})$ .

Da Hermite:  $P'(0) = R_1$ .

Logo:  $R_1 = 3 \cdot (P_{1\text{béz}} - P_1) \rightarrow P_{1\text{béz}} = P_1 + R_1/3$ . Analogamente para  $P_{2\text{béz}}$ .

## 3.7 Transformações Geométricas

### 3.7.1 Translação

$$P' = P + (dx, dy)$$

Aplicada a todos os vértices/pontos de controle. Para Hermite, os vetores tangente  $R_1$ ,  $R_4$  são **vetores relativos** e *não são transladados*.

### 3.7.2 Rotação

$$P' = C + R(\theta) \cdot (P - C)$$

$$R(\theta) = \begin{vmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{vmatrix}$$

Onde  $C$  é o centro de rotação. Para Hermite, os vetores tangente são rotacionados ao redor da **origem** (são vetores, não pontos).

### 3.7.3 Escala

$$P' = C + S \cdot (P - C), \quad S = \text{diag}(s_x, s_y)$$

Para Hermite, os vetores tangente são escalados proporcionalmente:  $R' = (R \cdot s_x, R \cdot s_y)$ .

#### 3.7.4 Reflexão

Implementada como escala com fatores negativos ( $s_x = -1$  ou  $s_y = -1$ ), decomposta em:

1. Des-rotacionar (alinhar eixos locais com eixos globais)
2. Aplicar reflexão
3. Re-rotacionar

## 4. Manual de Uso

### 4.1 Compilar e Executar

```
# Compilar todos os arquivos Java
javac -encoding UTF-8 *.java

# Executar a aplicação
java PaintApp
```

#### Requisito

Java JDK 8 ou superior instalado e disponível no PATH do sistema.

### 4.2 Barra de Ferramentas

A barra no topo da janela contém:

| Grupo      | Botões                                                                                                                                                                                                                                                                                                                                                   | Função                                          |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| Desenho    |     Béz Herm | Ponto, Reta, Círculo, Polígono, Bézier, Hermite |
| Seleção    |              | Selecionar, Pan, Rotação, Escala                |
| Reflexão   | X Y X Y                                                                                                                                                                                                                                                                                                                                                  | Reflexão em X, Y, XY                            |
| Algoritmos | DDA/Bres Cohen/Liang                                                                                                                                                                                                                                                                                                                                     | Toggle de rasterização e clipping               |

### 4.3 Desenhar Ponto

- 1 Selecione a ferramenta **Ponto** ()
- 2 Clique em qualquer posição do canvas
- 3 Um disco é criado na posição do clique

### 4.4 Desenhar Reta

- 1 Selecione a ferramenta **Reta** ()

- 2 **Pressione** o mouse na posição inicial
- 3 **Arraste** até a posição final (preview em tempo real)
- 4 **Solte** para confirmar a reta

## 4.5 Desenhar Círculo

- 1 Selecione a ferramenta **Círculo** (●)
- 2 **Pressione** para definir o centro
- 3 **Arraste** para definir o raio (distância do centro ao cursor)
- 4 **Solte** para confirmar

## 4.6 Desenhar Polígono

- 1 Selecione a ferramenta **Polígono** (□)
- 2 Clique para adicionar vértices
- 3 Quando houver 3+ vértices, clique próximo ao primeiro para **fechar** (aparece um círculo vermelho de hint)
- 4 Alternativamente, pressione V para validar o polígono aberto

## 4.7 Desenhar Curva de Bézier

- 1 Selecione a ferramenta **Béz**
- 2 Clique para definir **P<sub>0</sub>** (ponto inicial — a curva passa por aqui)
- 3 Clique para definir **P<sub>1</sub>** (1º ponto de controle — "puxa" a curva)
- 4 Clique para definir **P<sub>2</sub>** (2º ponto de controle — "puxa" a curva)
- 5 Clique para definir **P<sub>3</sub>** (ponto final — a curva passa por aqui)

#### Dica

Durante a construção, o MiniPaint mostra um **preview em tempo real**

da curva parcial. Os pontos de controle já colocados aparecem como discos verdes.

## 4.8 Desenhar Curva de Hermite

- 1 Selecione a ferramenta **Herm**
- 2 **Clique** para definir  $P_1$  (ponto inicial)
- 3 **Arraste** a partir de  $P_1$  para definir  $R_1$  (vetor tangente no ponto inicial — direção e magnitude da curvatura). Solte o mouse.
- 4 **Clique** para definir  $P_4$  (ponto final)
- 5 **Arraste** a partir de  $P_4$  para definir  $R_4$  (vetor tangente no ponto final). Solte o mouse para confirmar.

#### Dica

A magnitude do arraste controla a "intensidade" da curvatura: arrastar longe produz curvas mais amplas; arrastar pouco produz curvas mais suaves.

## 4.9 Seleção e Edição

- 1 Selecione a ferramenta **Selecionar** ( $\mathbb{R}$ )
- 2 Arraste um retângulo ao redor das shapes desejadas
- 3 As shapes totalmente contidas ficam **destacadas em vermelho** e o modo muda para **EDIT**
- 4 No modo EDIT, arraste para **mover** as shapes selecionadas

## 4.10 Transformações

| Transformação      | Como Usar                                                                                                                                                                               |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Translação</b>  | No modo EDIT, arraste as shapes selecionadas                                                                                                                                            |
| <b>Rotação</b>     | Com shapes selecionadas, clique em  e arraste. O centro de rotação é o centro do retângulo de seleção. |
| <b>Escala</b>      | Com shapes selecionadas, clique em  e arraste para dentro/fora do centro.                              |
| <b>Reflexão X</b>  | Clique no botão <b>X</b> — espelha horizontalmente ao redor do centro da seleção.                                                                                                       |
| <b>Reflexão Y</b>  | Clique no botão <b>Y</b> — espelha verticalmente.                                                                                                                                       |
| <b>Reflexão XY</b> | Clique no botão <b>XY</b> — espelha em ambos os eixos (equivalente a rotação de 180°).                                                                                                  |
| <b>Pan</b>         | Clique em  e arraste o canvas inteiro.                                                                 |

## 4.11 Atalhos de Teclado

| Tecla | Ação                                                               |
|-------|--------------------------------------------------------------------|
| V     | Valida/confirma o polígono aberto (sem fechar no primeiro vértice) |
| X     | Cancela desenho em andamento <b>ou</b> exclui shapes selecionadas  |

### Importante

Para que os atalhos funcionem, o canvas precisa ter foco. Clique no canvas antes de pressionar as teclas.